

mxfp4 scale pre-shuffling in Triton

1 Background of dot_scaled

The high-level algorithm of scaled dot operation is shown below. In addition to input A and B tensor, it also has scale tensors used for dequantization, which uses one scale value for 32 consecutive elements along the K dim. Scales are e8m0 and input values are mxfp4, i.e. e2m1. For more details regarding micro-scaling data formats, check <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>.

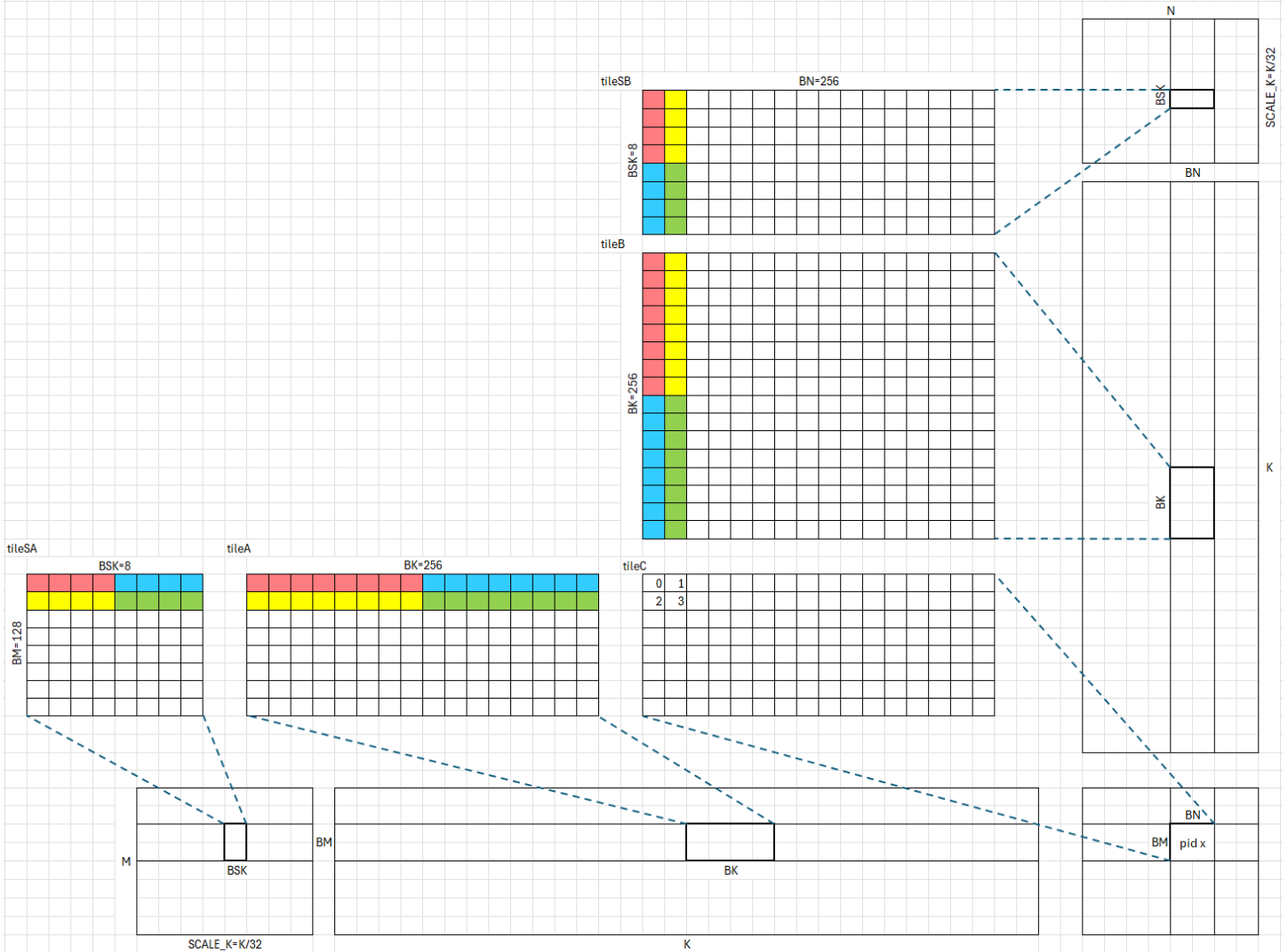


Figure 1. Overview of scaled dot operation

Implementation wise, scaled dot is very similar to regular dot kernels. The output tensor is partitioned among pids. There is a for loop to load blocks of data from both inputs and scaled to compute the scaled dot and accumulate into the results.

1.1 Terms

Symbol	Short name	Definition
M, N, K	M, N, K	Size of the inputs
SCALE_K	SK	$K / 32$
BLOCK_M	BM	Block size along the M dim for input A
BLOCK_N	BN	Block size along the N dim for input B
BLOCK_K	BK	Block size along the K dim for input A and B
BLOCK_SCALE_K	BSK	Block size along the K dim for scale A and B = SCALE_K / 32
tileA, tileB	tileA, tileB	One block from input A and B in the loop

tileSA, tileSB	tileSA, tileSb	One block from scale A and B in the loop
tileC	tileC	One output block
Basic tile	BT	The minimum tile that one wave needs to load
Basic scale tile	BST	The minimum scale tile that one wave needs to load

2 The need for scale preshuffling

In Figure 1, it uses an example tile size of BM=128, BN=256, BK=256. Therefore, BSK=8. Each cell in tileC represents a 16x16 fp32 tensor. As an example, cell0-4 of tileC is computed as

$$scaleXDL(colorA, colorB) = scaledMFMA(tileA_{colorA}, tileSA_{colorA}, tileB_{colorB}, tileSB_{colorB})$$

$$tileC_0 = scaledXDL(red, red) + scaledXDL(blue, blue)$$

$$tileC_1 = scaledXDL(red, yellow) + scaledXDL(blue, green)$$

$$tileC_2 = scaledXDL(yellow, red) + scaledXDL(green, blue)$$

$$tileC_3 = scaledXDL(yellow, yellow) + scaledXDL(green, green)$$

Let's take a closer look at the layout of scale A and input A involved in the above equations, as shown in Figure 2.

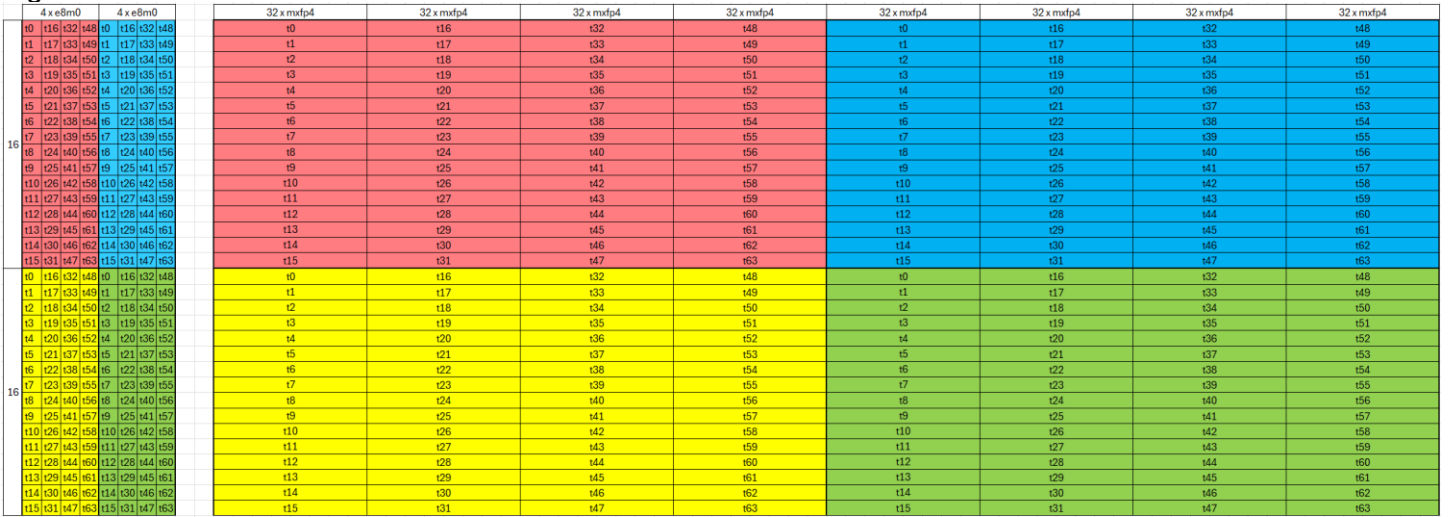


Figure 2. Operand and scale A layout of 4 adjacent scaled mfma instructions

Each mfma_scale_16x16 instruction computes a scaled dot operation of size 16x16x128, which means the input A tile is of shape 16x128xf4 and scale A is of shape 16x4xe8m0. The red part of Figure 2 shows the element and thread layouts of input A and scale A of a single mfma_scale_16x16 instruction. The input A and scale A layouts are the same as dotOperand layout. Note that for scale A layout, kWidth=1, meaning each thread has 1 scale value. Since scale values are 8-bit and each thread is assigned at least one 32-bit register to hold the values, 24 bits out of 32 bits are wasted.

2.1 Why at least 4 scale values?

To fully utilize the register for scale values, each thread needs to hold at least 4 scale values so that they can be put into one 32-bit register. For a single mfma_scale instruction, scale A register (32-bit) has 4 scale values and the *op_sel* flag is used to indicate which of the 4 values (8 bits) is used for the current mfma_scale instruction.

Putting 4 scale values of the same thread together can also benefit load efficiency. For example, when loading scale values from LDS, the kernel can use *ds_read_b32* to load 4 scale values at once.

2.2 Which 4 scale values?

We learned that each thread needs to put 4 scale values together to take up one 32-bit register. Now we need to figure out where these 4 scale values come from. Figure 2 is one example, in which the 4 scale values come from the 32x256 input A tensor.

There are other options to choose the 4 scale values, such as Figure 3 and Figure 4.

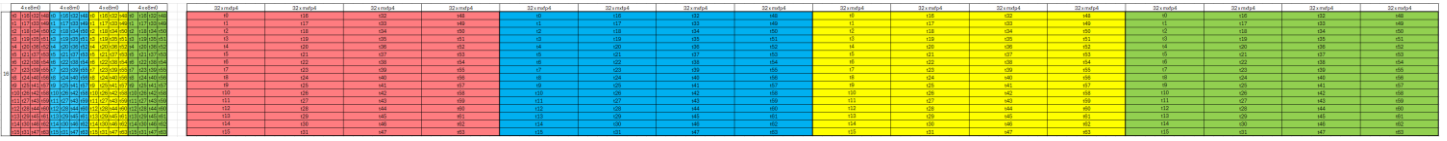


Figure 3. 4 scale values coming from 16x512 input.

4 x 64x128		32 x mxfp4			
16	10	116	132	148	164
	11	117	133	149	165
	12	118	134	150	166
	13	119	135	151	167
	14	120	136	152	168
	15	121	137	153	169
	16	122	138	154	170
	17	123	139	155	171
	18	124	140	156	172
	19	125	141	157	173
	20	126	142	158	174
	21	127	143	159	175
	22	128	144	160	176
	23	129	145	161	177
	24	130	146	162	178
	25	131	147	163	179
16	10	116	132	148	164
	11	117	133	149	165
	12	118	134	150	166
	13	119	135	151	167
	14	120	136	152	168
	15	121	137	153	169
	16	122	138	154	170
	17	123	139	155	171
	18	124	140	156	172
	19	125	141	157	173
	20	126	142	158	174
	21	127	143	159	175
	22	128	144	160	176
	23	129	145	161	177
	24	130	146	162	178
	25	131	147	163	179
16	10	116	132	148	164
	11	117	133	149	165
	12	118	134	150	166
	13	119	135	151	167
	14	120	136	152	168
	15	121	137	153	169
	16	122	138	154	170
	17	123	139	155	171
	18	124	140	156	172
	19	125	141	157	173
	20	126	142	158	174
	21	127	143	159	175
	22	128	144	160	176
	23	129	145	161	177
	24	130	146	162	178
	25	131	147	163	179
16	10	116	132	148	164
	11	117	133	149	165
	12	118	134	150	166
	13	119	135	151	167
	14	120	136	152	168
	15	121	137	153	169
	16	122	138	154	170
	17	123	139	155	171
	18	124	140	156	172
	19	125	141	157	173
	20	126	142	158	174
	21	127	143	159	175
	22	128	144	160	176
	23	129	145	161	177
	24	130	146	162	178
	25	131	147	163	179

Figure 4. 4 scale values coming from 64x128 input.

We define the chosen input as a basic tile (BT) and the corresponding scale tile as a basic scale tile (BST). Note that the definition of BT is slightly different from the one defined in v2¹. Here BT and BST have mxfp4 input layouts, and BST can be loaded by one load operation. However, BT needs to be loaded by 4 load operations. We call it a basic tile because waves always load all or none of the elements in the basic tile.

Which 4 scale values to choose determines the shapes of the BT and BST. The shapes in Figure 3 (16x512 for BT and 16x16 for BST) require BLOCK_K to be at least 512, which is not the optimal BLOCK_K size for most of the dense matmul kernels. Similarly, the shapes in Figure 4 (64x128 for BT and 64x4 for BST) require BLOCK_M or BLOCK_N to be at least 64, which is larger than M or N size for most decode matmul kernels. Therefore, **the shapes in Figure 2, 32x256 for BT and 32x8 for BST, are chosen.**

¹ [Pre-sshuffling_v2.pdf](#)

2.3 What order of the 4 scale values?

We have chosen 4 scale values for each thread to put into a 32-bit register. To get the best performance, the 4 scales assigned to a single thread need to be contiguous in memory so that the load operation for the BST can be vectorized.

In Figure 2, the 4 scale values assigned to each thread are not contiguous. Therefore, **we need to shuffle the scales so that the 4 scale values for each thread are contiguous in memory.**

Now the last question is: how to order the 4 scale values?

There are multiple options in Figure 2, e.g.

1. red --> blue --> yellow --> green
2. red --> yellow --> blue --> green
3. red --> green --> yellow --> blue

There are no pros and cons for each option. The only requirement is that the shuffling algorithm and the compiler agree on the same option.

The framework team decides to use a column-major order, i.e. red --> yellow --> blue --> green.

3 The scale shuffling algorithm

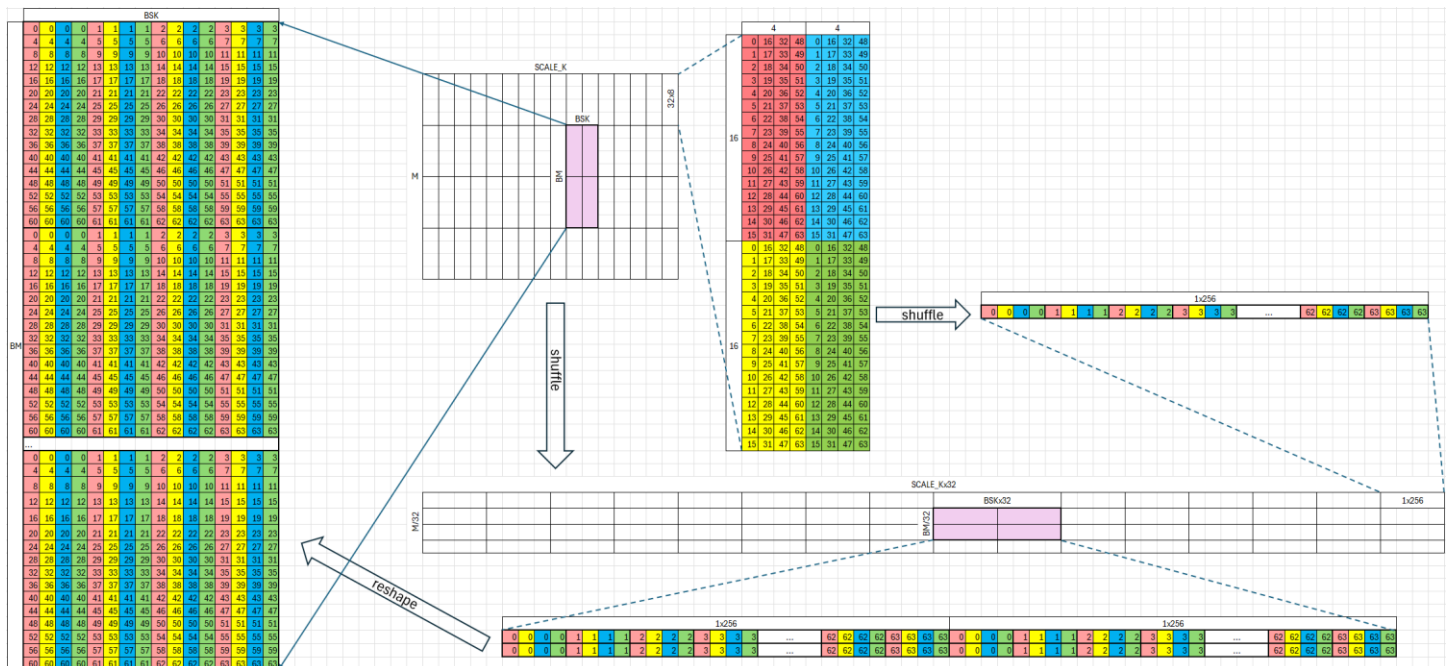


Figure 5. Overview of the scale pre-shuffling algorithm.

Figure 5 illustrates the overview of the scale pre-shuffling algorithm. The general idea is to partition the scale into BSTs of shape 32x8. Shuffle the elements in each BST so that the 4 scale values per thread are contiguous and the result BST becomes 1x256. The layout of BSTs has not changed. The shuffled scale tensor has a different shape. When the kernel load scale tiles, the tile size needs to be adjusted, and the loaded scale tile needs to be reshaped to the original shape to feed the dot_scaled op. Finally, the compiler needs to figure out the correct scale value and use the proper *op_sel* flag when generating mfma_scale instructions.

3.1 Torch Support

The scale tensor, allocated as an M by SCALE_K tensor as an example, needs to be shuffled by the following algorithm²

² Check [Pre-shuffling_v1.pdf](#) (section "Pre-shuffling algorithm") for a detailed explanation of view, permute, and reshape.

Algorithm 1. Scale tensor shuffling

```

scale = scale.view(M // 32, 2, 16, SCALE_K // 8, 2, 4, 1)
scale = scale.permute(0, 3, 5, 2, 4, 1, 6)
scale = scale.reshape(M // 32, SCALE_K * 32)

```

3.1.1 Shuffling within BST

The original scale tensor is partitioned into BSTs of shape 32x8. Each BST is shuffled into a 1D tensor s.t.

1. the 4 scale values per each thread are contiguous
2. values from threads are in thread id ascending order
3. values within each thread are in column-major (red-yellow-blue-green) order

The shuffling of each BST is illustrated in Figure 6.

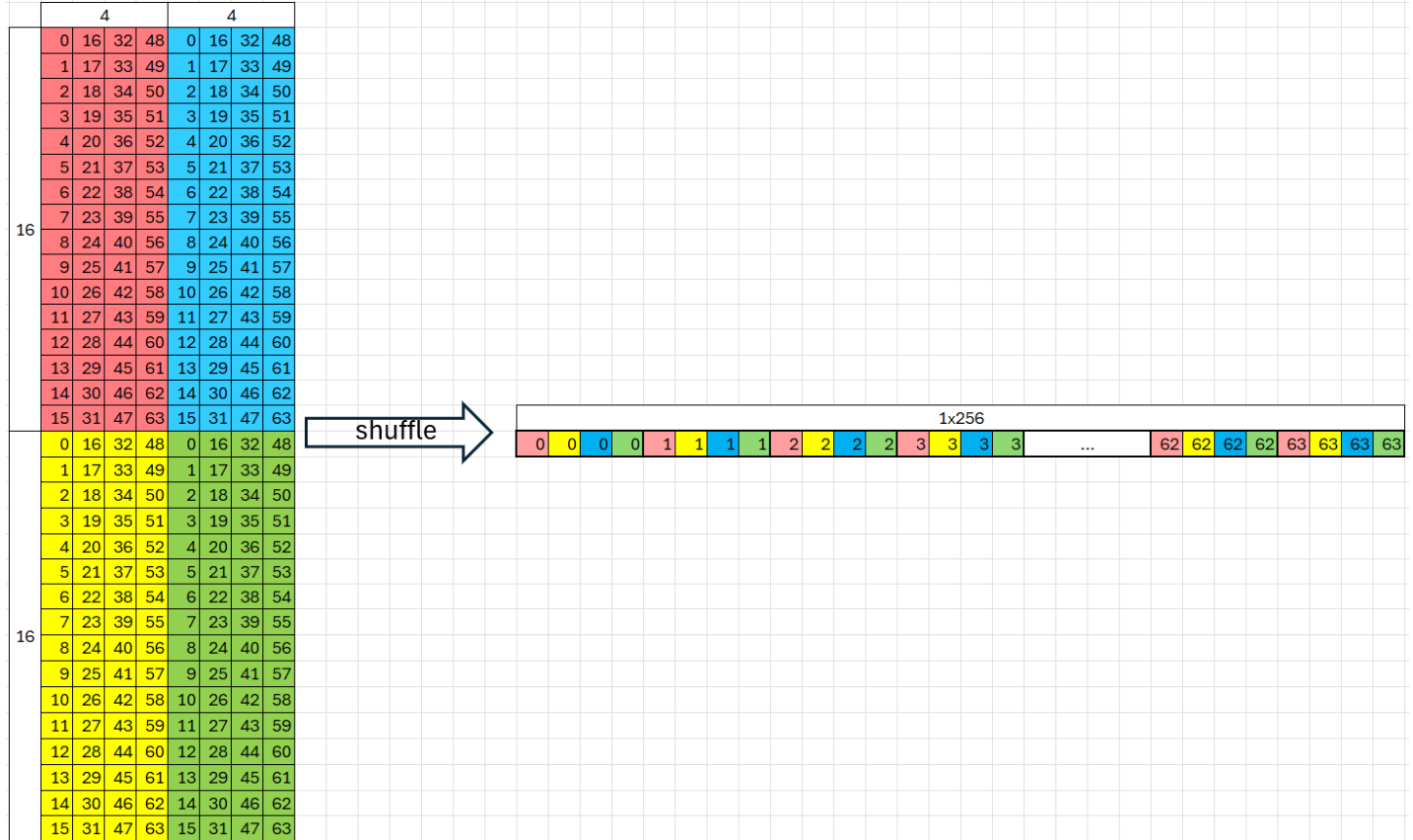


Figure 6. Shuffling algorithm of each 32x8 BST.

3.1.2 Shuffling of BSTs

The layout of the BSTs stays the same, i.e. neighboring BSTs (all 4 directions) in the unshuffled scale tensor are still neighbors in the shuffled tensor, as shown in Figure 7.

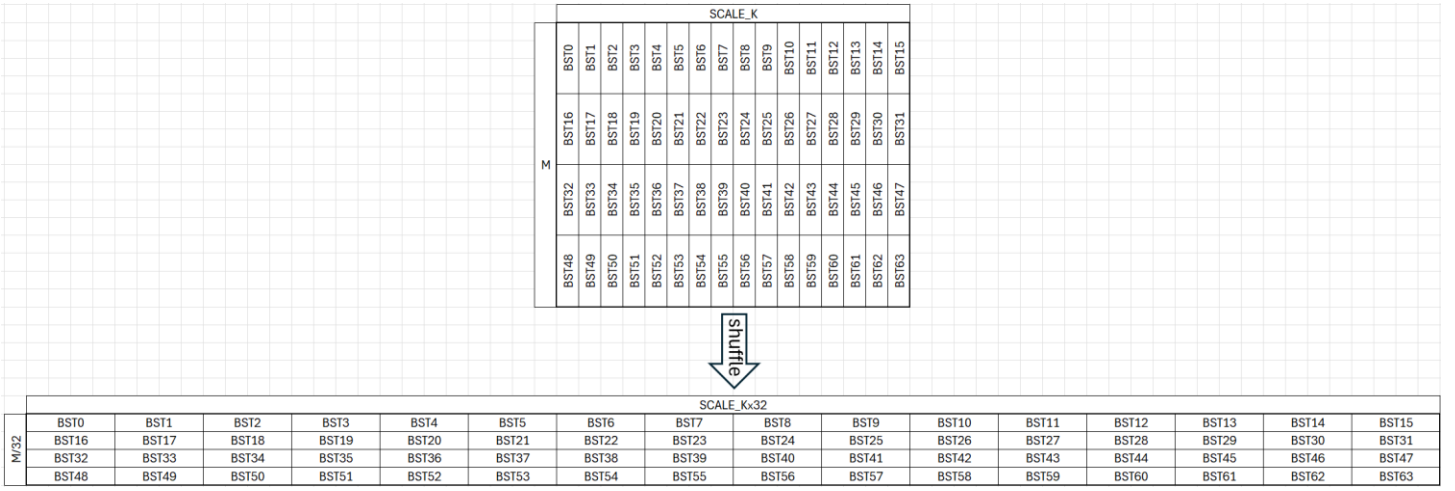


Figure 7. Illustration of BST layouts before and after scale tensor shuffling.

3.1.3 Reshape the scale tensor

The last step of the shuffling algorithm is to reshape the scale tensor into $M/32$ by $SCALE_K * 32$. Note that we cannot reshape the scale tensor to its original shape, or any other shapes, since the 2D geographical relations between the BSTs must be preserved. This is because when the kernel accesses a tile of scale tensor in the loop, it accesses a block covering several BSTs. We need to guarantee the BSTs covered by a block before shuffling can also be covered by a block after shuffling.

A counter example is shown in Figure 8, in which the scale tensor is reshaped into $M/16$ by $SCALE_K * 16$. As an example, assume before scale shuffling, one workgroup needs to load BST25,26,41,42 for one iteration of the loop, as highlighted in Figure 8. However, in the shuffled version, those 4 BSTs are not adjacent. Thus, the workgroup cannot load the required BSTs to do the computation.

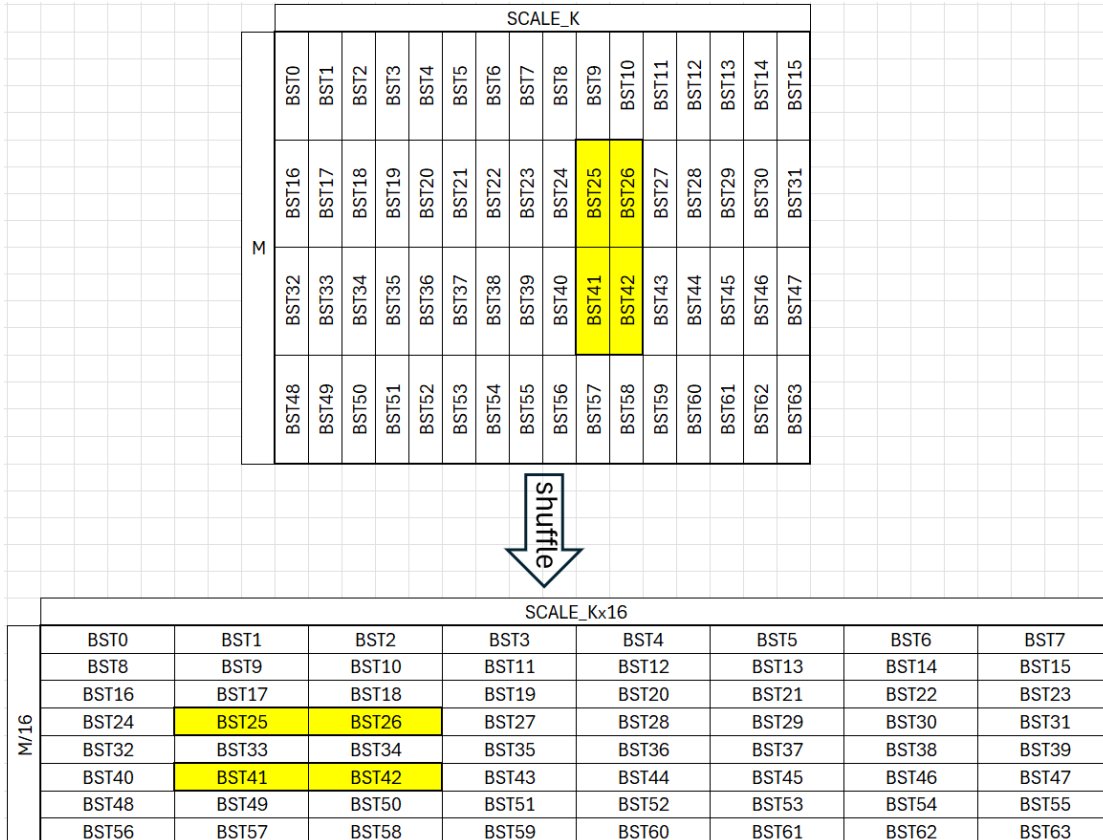


Figure 8. An illustration of wrong shape after reshaping.

Note that this is a limitation for Triton, but not for CK or hipBLASlt kernels. Triton kernel is written under the workgroup scope, and the programming model is based on tile level.

3.2 Kernel Support

As mentioned in the previous sub-section, the scale tensor of shape M by SCALE_K is shuffled and reshaped into M/32 by SCALE_K*32. In the kernel, we need two changes as explained in the following.

3.2.1 Locate the correct tile

The pointer arithmetic used to calculate the addresses of the tile needs to be adjusted accordingly. More specifically, when calculating the start offset and the ranges of the tile, it needs to use BLOCK_M/32 and BLOCK_SCALE_K*32 as the tile size since each tile is reshaped from BLOCK_M by BLOCK_SCALE_K into BLOCK_M/32 by BLOCK_SCALE_K*32.

3.2.2 Reshape the tile

The dot_scale op has requirements on the shape of its inputs. It requires the shape of the scale tensor to be BLOCK_M by SCALE_K for an input of shape BLOCK_M by SCALE_K*32. Therefore, the loaded scale tile needs to be reshaped by tl.reshape as shown in Figure 9.

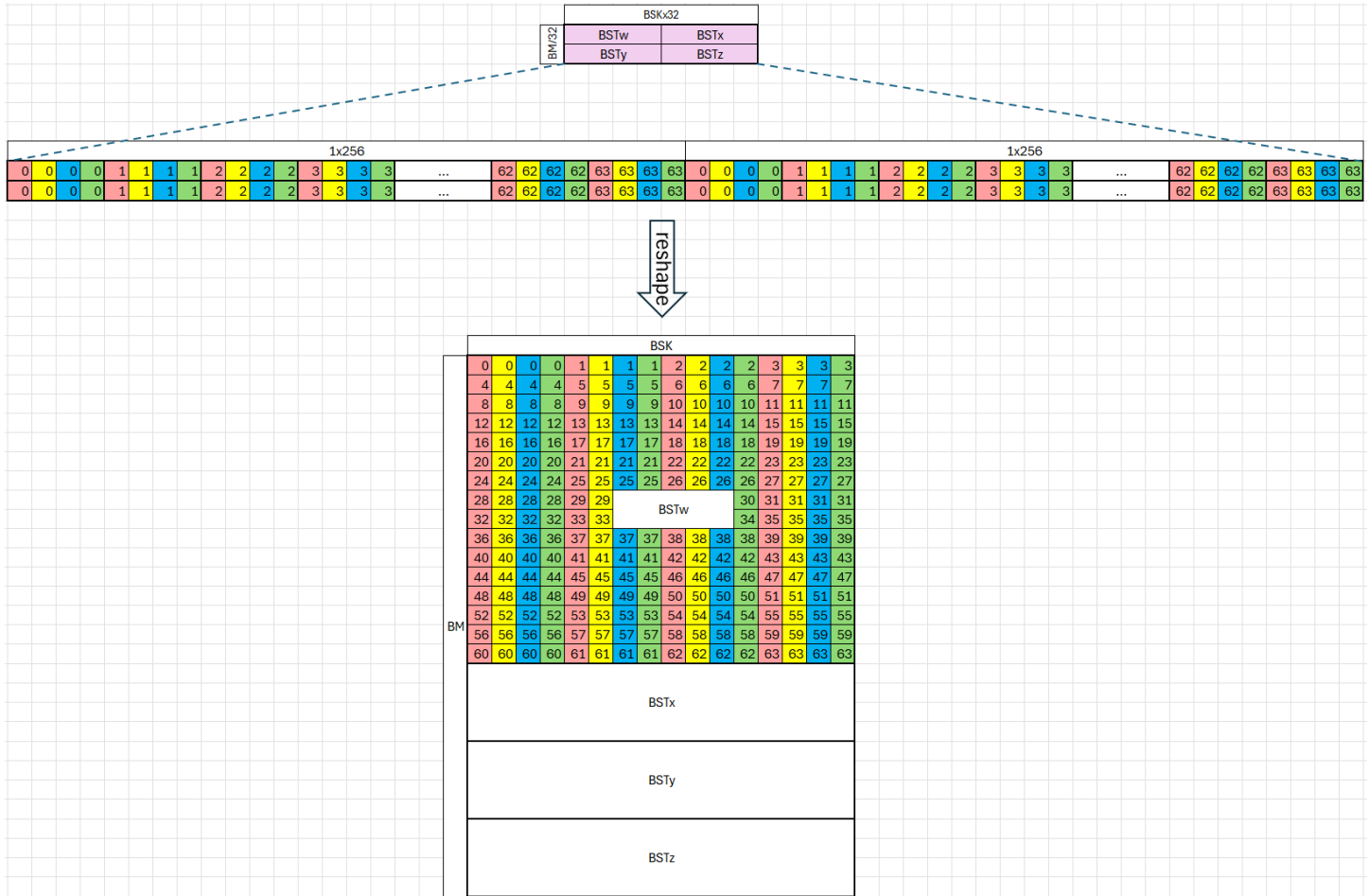


Figure 9. Illustration of reshape of loaded scale tile from BM/32 by BSK*32 to BM by BSK.

The reshape operation changes the thread layout only. Each thread still holds the same elements from the scale tensor. Since there is no data exchange between threads, the reshape op is a free op, which means the compiler does not generate any assembly instruction out of it.

Note that at Triton kernel level, layout is not exposed. Such information is used in the compiler, which is explained later.

4 Compiler Support for scale shuffling

In previous sections, we talked about torch and kernel level support to shuffle the scale tensors. To illustrate how shuffling is done, the layout of elements and BSTs are shown. However, layout is only used by the compiler and it's the compiler's responsibility to implement such layout so that the scaled dot operation produces correct results.

4.1 dot_scaled Layout

For the dot_scaled op, the compiler chooses the input and output layout. The chosen layout should match the shuffling algorithm applied at torch and kernel level.

4.1.1 Warp Layout (Output Layout)

In each iteration of the k loop, the kernel computes the dot scaled operation from tileA, tileB, tileScaleA and tileScaleB and produces tileC, as shown in Figure 10. Due to the requirement of BT and BST, each wave needs to compute an output tile of 32x32 at minimum. The output of a single mfma_scale_16x16 is 16x16, therefore each wave needs to compute 4 output tiles as a unit. In Figure 10, the output tiles assigned to wave0 (w0) is highlighted in purple. The layout of the output tile is referred to as MfmaLayout in Triton layout system³. However, the output layout shown in Figure 10 is not an MfmaLayout, because each wave computes multiple adjacent 16x16 output tiles. Therefore, we extend the current MfmaLayout by adding a **tilesPerWarp** parameter ([code](#)), indicating the adjacent tile layout for each wave⁴. Once the MfmaLayout is resolved, the wave layout for inputs and scales can be resolved accordingly. The inputs and scales needed by wave 0 are also highlighted in Figure 10.

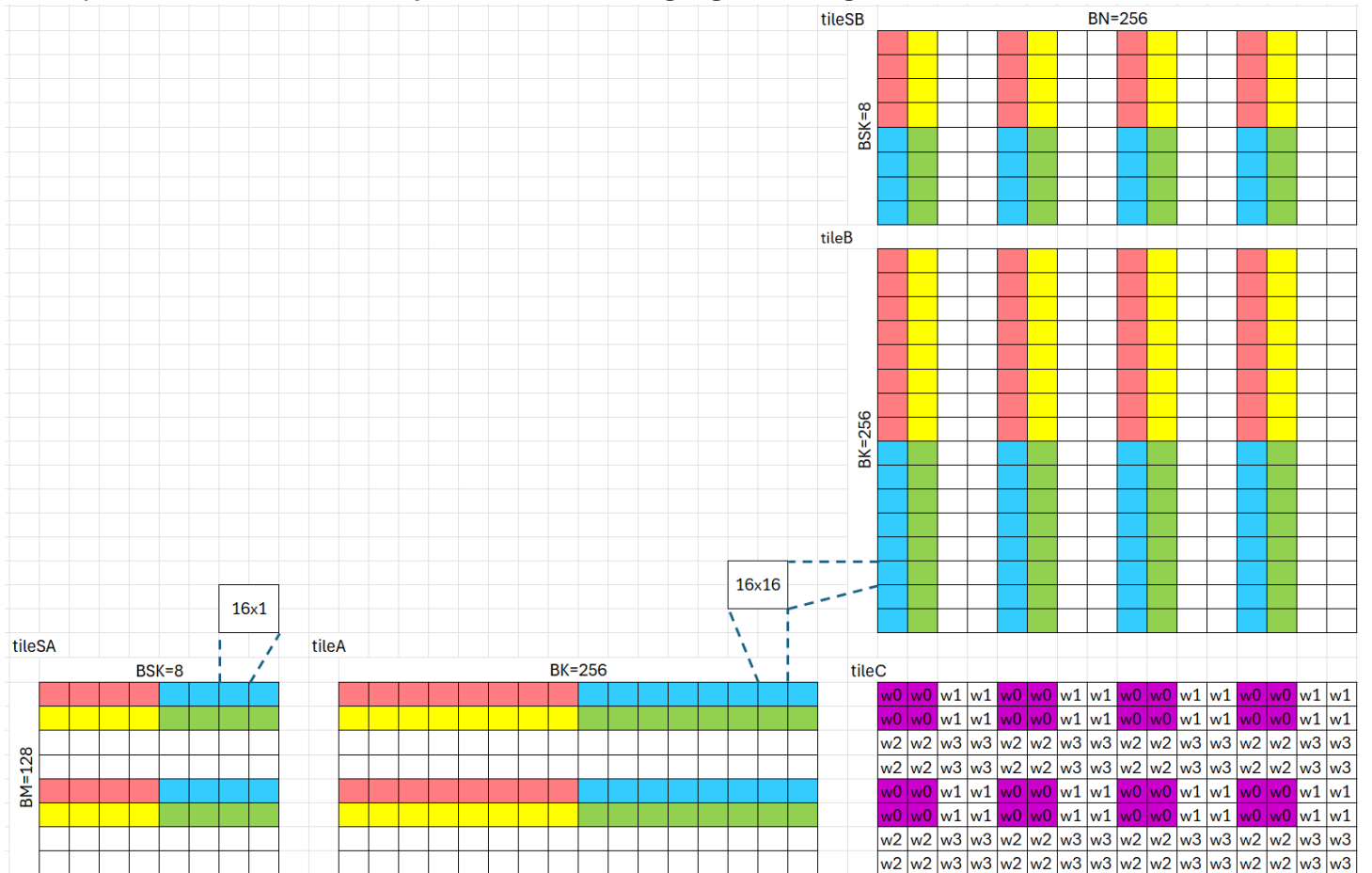


Figure 10. An illustration of dot_scaled op for 128x256x256 tile size.

³ [Triton_layout_introduction.pdf](#)

⁴ In the future, we will use linear layout to represent this output layout directly.

4.1.2 Element and Thread Layout (Input Layout)

Scale shuffling does not change the layout of the inputs. Thus, we focus on the layout of the scale tensor only. Figure 11 compares the scale layouts before and after the shuffling with an example scale tile size of 64 by 16 consisting of 4 BSTs.

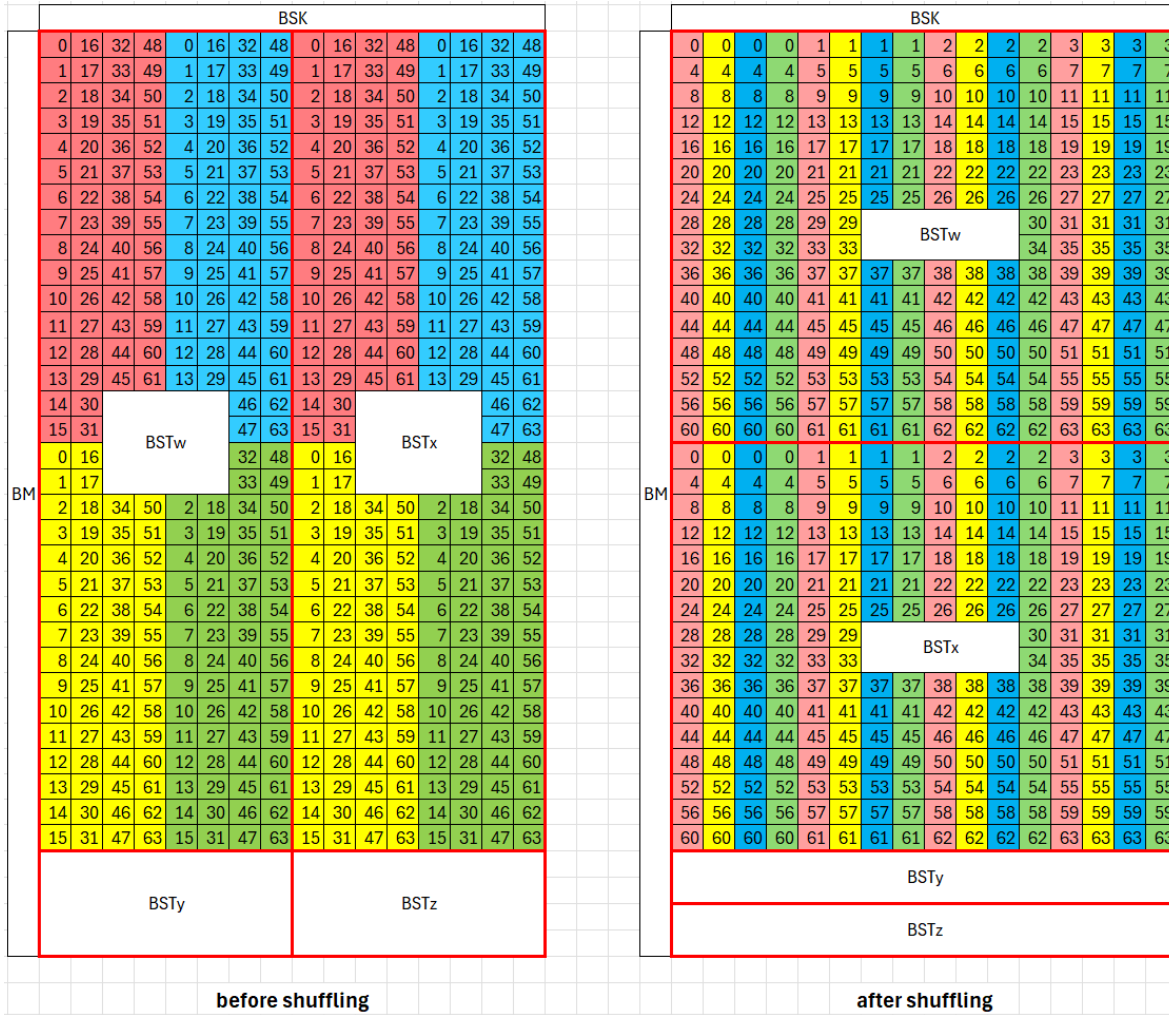


Figure 11. scale tile layouts before and after shuffling.

The number in each cell refers to the thread id that owns the element. Before shuffling, the layout of the scale tile is a repetition of 16x4 sub-tiles, each of which has a layout of

{reg: null; lane: (0,1), (0,2), (0,4), (0,8), (1,0), (2,0)}.

After shuffling, the layout of the scale tile is a repetition of 256/BSK by BSK sub-tiles, each of which has a layout of

{reg: (1,0), (2,0); lane: (4,0), (8,0),..., (log2(BSK/4)*4,0), (0,1), (0,2), ..., (0,log2(256/BSK))} ([code](#)).

4.2 mfma_scale_16x16 instruction generation

The compiler first prepares all values from the inputs and scales and then generates mfma_scale instructions with proper operands. Since the input and output layouts of the dot_scaled operation is resolved, here the compiler only needs to figure out the groups of values per thread associated with each mfma_scale instruction.

4.2.1 Operand Preparation

When the compiler starts to lower the dot_scaled op, the operands of dot_scaled op are in registers in the form of an array of values. The dot_scaled op is lowered to many mfma_scale_16x16 instructions.

The compiler takes the raw array of the operand values and packs them into vectors of `kBase` values. **`kBase` is defined as the number of values each thread needs for a single `mfma/mfma_scale` instruction.** During `mfma` instruction generation, one vector of `kBase` values per thread per operand is used to generate one `mfma_scale` instruction.

The operands are processed (loaded from global memory or LDS, results of another in-register op) into a 1D array of values. With `dotOperandLayout`, the values are assumed to be in the form of vectors of `kWidth` elements. **`kWidth` is a parameter of the `dotOperandLayout`, which indicates the number of contiguous elements each thread prepared** (loaded from global memory or LDS) as input to `dot` or `dot_scaled` ops.

Note that `kWidth` refers to the number of elements each thread prepared, which does not necessarily equal to `kBase`, which is the number of elements needed by a single `mfma` instructions. `kWidth` is usually limited by the maximum vectorization width of the load instructions, i.e. 128 bits for `buffer/global_load` and `ds_read`, 64 bits for `ds_read_tr`. `kBase` is defined according to the ISA. The idea of using `kWidth`, which could be different from `kBase`, is that threads can maximize the load (either from global memory or LDS) efficiency without worrying about hardware requirements on the users of the loaded data.

The relation between `kWidth` and `kBase` is summarized in Table 1.

Relation	Explanation	Example
<code>kWidth > kBase</code>	Each thread prepares more values than one <code>mfma</code> instruction can consume.	This is the case when <code>kpack</code> is set to 2 on MI300, i.e. <code>ds_read_b128</code> is used to load the operands. However, for <code>fp16/bf16</code> <code>mfma</code> instructions, each thread only needs 64 bits of data. Thus, one <code>ds_read_b128</code> corresponds to 2 <code>mfma</code> instructions.
<code>kWidth == kBase</code>	Each thread prepares the same number of elements that are needed for one <code>mfma</code> instruction.	This is the case for inputs with MI350, or when <code>kpack=1</code> on MI300.
<code>kWidth < kBase</code>	One vector of <code>kWidth</code> elements prepared by each thread is not enough to invoke one <code>mfma</code> instruction. Therefore, one <code>mfma</code> instruction consumes multiple vectors.	This is the case for the 2 nd dot operation in attention kernels with MI350 ⁵ . This is also the case for the scale tensor.

Table 1. Relations between `kWidth` and `kBase`.

With a clear knowledge of `kBase` and `KWidth`, let’s dive deep into how operands are prepared for `tileA` and `tileScaleA`. An example of tile size 128x256x256 is illustrated in Figure 12.

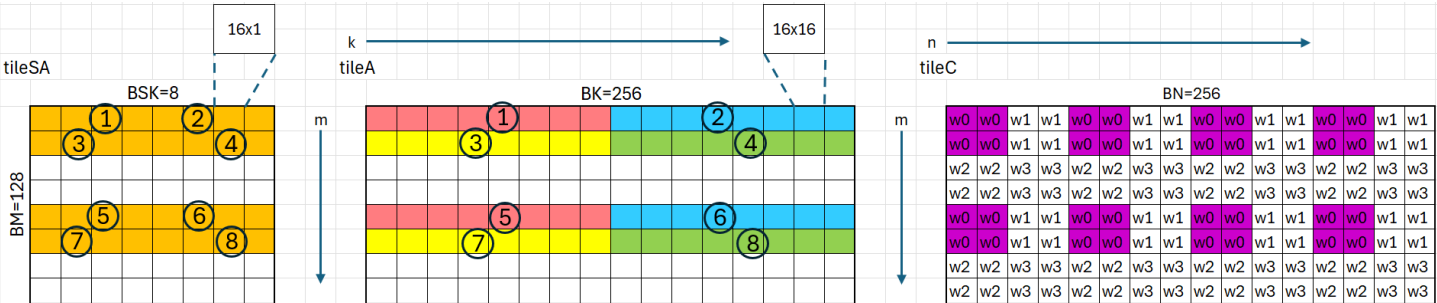


Figure 12. Illustration of operand preparation for input A and scale A.

⁵ [PR6540](#)

4.2.1.1. tileA (same for tileB)

From the output layout, it's clear that wave 0 (same for other waves) needs inputs of the colored areas in tileA. Usually, the colored area cannot be loaded by a single load instruction, therefore the colored area is partitioned into sections, each of which can be loaded by a single load instruction. The section dimension is determined by

1. kWidth, which decides the number of contiguous elements per thread along the fastest dimension
2. thread layout. For mfma_scale_16x16 instructions, the thread layout is {lane: (0,1), (0,2), (0,4), (0,8), (kWidth, 0), (2*kWidth, 0)}. Such a layout is inferred from the MfmaLayout (the output layout).

Therefore, the colored area is partitioned into 8 sections, as labeled from ① to ⑧ in Figure 12. The sections follow a k-contig order. Each section corresponds to a sub-tile of shape 16x128.

The raw values from tileA for each thread are in the format of numVecInKWidth (8) vectors of kWidth (32) elements. The compiler re-groups the elements into numVecInKBase (8) vectors of kBase (32) elements. The process is illustrated in Figure 13.

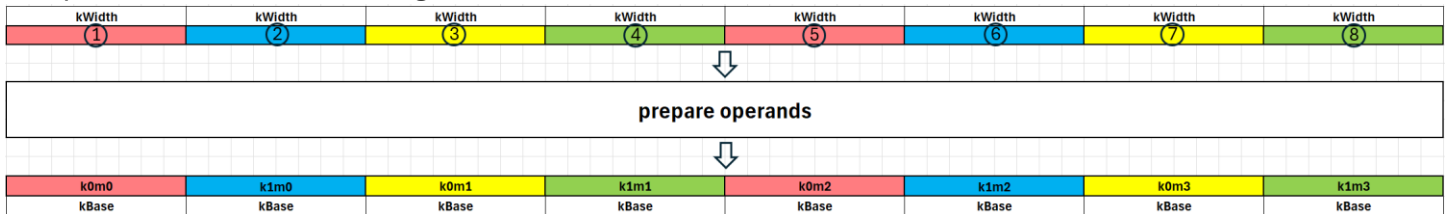


Figure 13. tileA operands preparation.

The result numVecInKBase vectors are saved in the tileAMap, whose key is the index (k,m) of the vector along k and m dimensions.

4.2.1.2. tileScaleA (same for tileScaleB)

The process of tileScaleA is like tileA, except tileScaleA is shuffled. However, the section partitioning logic follows the same one for tileA, which does not take shuffling into consideration. This means the compiler thinks the 4 consecutive scale values per thread are loaded one at a time, i.e. kWidth=1, instead of being loaded as a vector by a single load instruction (kWidth=4). This is not an issue for operands preparation for tileScaleA, since values are in register domain and it does not matter how they are loaded from memory or LDS.

One mfma_scale instruction takes 4 scale values from scaleA⁶. Therefore, kBase is set to 4. The operands preparation for tileScaleA is illustrated in Figure 14.

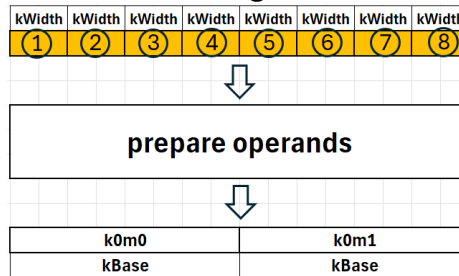


Figure 14. tileScaleA operands preparation.

After kWidth and kBase is set properly for tileScaleA, the process of preparing operands is the same as tileA. The raw values from tileScaleA for each thread are in the format of numVecInKWidth (8) vectors of kWidth (1) elements. The compiler re-groups the elements into numVecInKBase (2) vectors of kBase (4) elements.

⁶ It takes 4 values but only uses one of them at a time.

The result numVecInKBase vectors are saved in the tileScaleAMap, whose key is the index (k,m) of the vector along k and m dimensions.

4.2.2 Generate mfma instructions

In the previous section, the compiler has prepared operands in vectors of kBase elements. For any mfma_scale instruction, it needs one of these vectors for each operand. When generating mfma_scale instructions to compute the results in the output tile, the compiler

1. Traverses the output tiles going along n dim first. For each tile of index (m,n), the compiler
2. Traverses the iterations of the k dim. For each iteration of index k, the compiler
3. Uses indices (k,m) and (k,n) as the keys to query the vectors in the maps prepared for each operand and generate one mfma_scale instruction.

However, there is no 1:1 mapping from tileA vectors to tileScaleA vectors. This means for a given index (k,m), the compiler can always find a vector in the tileA map, but it may not find a vector in the tileScaleA map. This can be seen from the comparison of the outputs between Figure 13 and 14. This is because **the compiler groups scale values corresponding to 4 vectors of tileA into a single vector in the tileScaleA map**. Therefore, the mapping from tileA vectors to tileScaleA vectors is 4:1, i.e. 4 tileA vectors share the same tileScaleA vector. For these 4 tileA vectors, the op_sel flag is used to select different scale values from the shared tileScaleA vector. The mapping is illustrated in Figure 15.

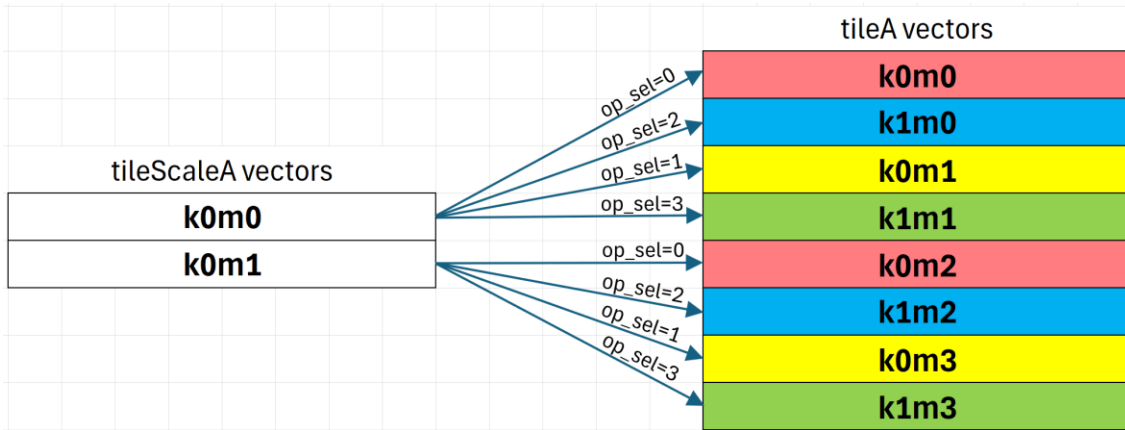


Figure 15. Illustration of vector mapping between tileA and tileScaleA with op_sel.

Note that the order of scale values follows the red-yellow-blue-green order defined in section 2.3.

Algorithm 2 presents how to compute tileScaleA index (kScaleA, mScaleA) and op_sel_A values given tileA index (k, m). The same algorithm is used to compute the indices for tileScaleB.

Algorithm 2. Compute tileScaleA (k,m) and op_sel values given index (k,m) for tileA

Input: tileA vector index (k, m), numVecInKBase for tileA

Output: tileScaleA vector index (kScaleA, mScaleA) and op_sel_A

```
tileId = m / 2 * (2*numVecInKBase) + m % 2 + k * 2
```

```
op_sel_A = tileId % 4
```

```
newKSize = numVecInKBase / 4
```

```
tileLeaderId = tileId / 4
```

```
mScaleA = tileLeaderId / newKSize
```

```
kScaleA = tileLeaderId % newKSize
```

Now we have all the pieces, the compiler can generate mfma_scale instruction for index (m,n,k) with the following

```
mfma_scale(tileAMap[k, m], tileBMap[k, n], tileScaleAMap[kScaleA, mScaleA],
           tileScaleBMap[kScaleB, nScaleB], opSelA, opSelB)
```

Where kScaleA, kScaleB, mScaleA, nScaleB are computed according to Algorithm 2.